Android Content		
Days	Topic	Details
1	Introduction to fundamentals	What is Software? Types of Software with examples & purpose • What is Android? Brief history, versions, and tools evolution • Android OS Architecture – Layers and their roles • Android SDK Setup and Studio Installation • Hands-on: Install Android Studio, create Hello World app • Demo: Hello World Android App (first practical exercise)
2	UI basics and XML	What is XML? Role in Android development • Hands-on: Create Simple Login Screen using XML • Introduction to Activities – Understanding Android Java file structure • Hands-on: Add basic functionality with Java • Understanding project structure and resources
3	Core Android Concepts	Activity Life Cycle – Explanation with example • Practice session: Students replicate previous builds • AndroidManifest.xml – Purpose and uses • build.gradle – Structure and importance • WebView • Q&A / Doubt clearing session
4	Intents & Navigation	What are Intents? Types (Explicit and Implicit) • Hands-on: Switching between Activities • Hands-on: Sharing Data between Activities using Intents • AndroidManifest.xml & Permissions – Declaration and usage • Handling runtime permissions
5	App Packaging & Components	• APK vs AAB – Components & functionality • App Bundles – Purpose and importance • Hands-on: Using bundletool to inspect and convert AABs • Basic building blocks: Services overview • Basic building blocks: Broadcast Receivers overview • Basic building blocks: Content Providers overview
6	Debugging & Logging	• Debugging Android projects – Breakpoints, step execution • Logcat – Syntax, usage, types • Hands-on: Using Logcat, breakpoints, inspecting runtime • Hands-on debugging examples • Review of concepts covered so far
7	Weekly Assessment 1	
8	Android Components Deep Dive	• Building apps with Activities – Advanced scenarios • Building apps with Services – Foreground and Background services • Building apps with Broadcast Receivers – System and custom broadcasts • Hands-on: Create app integrating all three components • Component communication patterns

9	Permissions, Manifest file and URIs	Accessibility Services Read SMS Write SMS Receive SMS Read Contacts Write Contacts ACCESS WIFI STATE ACCESS NETWORK STATE INTERNET CALL LOGS BIND NOTIFICATION LISTENER DEVICE ADMIN CALL PHONE WAKE LOCK
10	Introduction to Kotlin	• Kotlin Syntax basics – Variables, functions, classes • Kotlin Interoperability with Java • Converting Java code to Kotlin using Android Studio • Hands-on: Convert existing Java activity to Kotlin • Kotlin advantages for Android development
11	Kotlin Core Features	• Null Safety in Kotlin – Safe calls, Elvis operator • Smart Casting – Type checking and automatic casting • Hands-on: Build Kotlin app with null-safe logic • Hands-on: Refactor Java app to use Kotlin null safety • Data classes and sealed classes
12	Milestone Assessment 2	
13	Kotlin Advanced Features	• Lambdas and Higher-order functions • Extension Functions – Creating and using them • Hands-on: Refactor app using Kotlin idioms • Hands-on: Apply lambdas and extensions to existing code • Scope functions (let, apply, run, with, also)
14	Coroutines & Async Programming	• Introduction to Coroutines – Concepts and benefits • Coroutine builders and scopes • Implementing background tasks using coroutines • Hands-on: Network call simulation with coroutines • Flow and State Flow basics • Async Tasks
15	IPC:Intents & Services	• IPC mechanisms overview • Hands-on: Build two apps communicating via Intents • Hands-on: Build two apps communicating via Services • Hands-on: Data sharing between apps using Intents • Service binding and lifecycle
16	IPC: Advanced Communication	• Bound Services & AIDL – Concepts and implementation • Implementing Messenger & Binder communication • Hands-on: Create bound service with AIDL interface • IPC: Content Providers – CRUD operations • Hands-on: Build custom Content Provider
17	Native Libraries & JNI	• Native Libraries overview – What and why • JNI basics – Java Native Interface • Inspecting .so files in APKs • Using Frida for hooking native methods • Hands-on: Simple JNI example with native code • Security implications of native code and introduction to native Development Kit(NDK)

18	Weekly Assessment 2	
19	Jetpack compose	Composable functions Layouts (Row, Column, Box, LazyColumn) State management Event handling (clicks, text input) Modifiers Navigation Side effects (LaunchedEffect) ViewModel + StateFlow integration Interoperability (AndroidView)
20	Introduction to APIs	Retrofit, okhttp
21	Adb commands	Basic Commands Finding logs using ADB Interaction with emulator Emulator Rooting
22	Firebase concept	Realtime Database JSON tree structure How read/write operations work How Firebase paths look (users/, config/, cmd/) Understand these APIs: FirebaseDatabase.getInstance(), getReference(), child(), addValueEventListener(), addListenerForSingleValueEvent() google-services.json Know where it is located inside APK Read the following fields: project_id, storage_bucket, mobilesdk_app_id, api_key Firebase imports in APK Understand Firebase storage Firebase Cloud Messaging (FCM)
23	Milestone Assessment 3	
24	Basics of Framework: React Native, Cordova, Flutter,Cocos	Flutter - Dart code compilation (AOT) → Entire logic is compiled into libapp.so React Native - JavaScript code bundled inside APK:assets/index.android.bundle, Where native functionality is mapped (permissions, SMS, Contacts) Apache Cordova - Cordova uses HTML + JS + CSS → packaged as a web app Cocos / Cocos2d-x - Game loop architecture, Where assets are stored:assets/, APK contains native libraries:libcocos2d.so
25	Project Setup	• Project requirements: Building and securing a Kotlin-based Android app • Architecture planning (MVVM/Clean Architecture) • Security features implementation planning • Testing strategy definition • Project work: Students begin implementation • Instructor guidance and code reviews

26	Project Development	• Continued project development time • Implementing core features with Kotlin • Applying security best practices • Writing comprehensive tests • Code walkthrough and debugging challenge sessions • One-on-one mentoring and troubleshooting • Final project completion • Presenting app architecture, security features, and test results • Demonstrating working applications • Code review and feedback sessions • Assessment of learning outcomes • Closing remarks
27	Project Evaluation	